

SESSION 6

1. Count each user's orders

GROUP BY makes one group per user_id. COUNT(*) counts orders in that group.

```
SELECT user_id, COUNT(*) AS order_count
FROM orders
GROUP BY user_id;
```

2. Restaurants with average rating above 4

HAVING filters groups after AVG is calculated.

```
SELECT name, AVG(rating) AS average_rating
FROM restaurants
GROUP BY name
HAVING AVG(rating) > 4.0;
```

3. Payment subquery

The inner query finds qualifying user IDs; the outer query can retrieve them or join to Users for names.

```
SELECT user_id FROM payments
WHERE user_id IN (SELECT user_id FROM payments WHERE amount > 2000);
```

4. Movies above overall average

The nested subquery calculates the overall average. Each movie is then compared with it.

```
SELECT name, AVG(rating) AS movie_average
FROM movies
GROUP BY name
HAVING AVG(rating) > (SELECT AVG(rating) FROM movies);
```